



SPRING 2026

SCALABLE AI SERVICES WITH AGENTIC AI, MLOPS & LLMOPS

CS5305/CS621 - PROGRAMMING ASSIGNMENT 4

CS5305/CS621
SPRING 2026
LUMS SBASSE



Advanced LangGraph State Machine: Game Development

Assignment Overview

In this lab, you will design and implement a multi-agent AI system that collaboratively builds a fully functional Dino Runner game (Chrome Dino style) through an iterative development loop. The system simulates a real-world software development pipeline involving multiple specialized agents interacting to write, test, and refine code.

The Development Workflow:

1. **Director:** Provides the initial prompt and objective (e.g., "Build a Dino Runner game").
2. **Architect:** Breaks the request down into actionable system design components.
3. **Engineer:** Translates the architect's design into Python game code.
4. **Execution Manager:** Saves the code and, pending human approval, runs it.
5. **QA Engineer:** Analyzes the execution output, code, and requirements to identify errors or required features.
6. **Scorer:** Summarizes QA feedback into a quantitative metric (1–10).
7. **Human-in-the-Loop:** Evaluates the score and decides whether to trigger a new iteration (sending the flow back to the Engineer) or finalize the project.

Key Technologies & Concepts

- LangGraph
- Human-In-the-Loop
- Reducers
- Graph State: Nodes, Edges, & Conditional Edges

Prerequisites

Task 0: Workspace Initialization

Requirement: You MUST utilize Mosaic AI resources to complete this assignment. Ensure your environment is properly initialized before proceeding.

Part 1 [100 marks]

Task 1: Initializing System Memory and State Management [10 marks]

Objective: Define a persistent state structure that maintains memory across all agent interactions.

- Note: You must use the `add_messages` reducer to ensure historical context is preserved rather than overwritten.

Requirements:

Define a `GameState` dictionary containing the following exact fields:

1. `director_messages`: Stores all user/director prompts
2. `architect_messages`: Stores system design outputs from the architect
3. `engineer_code`: Stores generated Python game code for each iteration
4. `qa_feedback`: Stores QA analysis reports for each iteration
5. `current_actor`: Tracks which agent is currently active
6. `iteration`: Tracks iteration count of the system
7. `iteration_score`: List of numeric scores (1–10) per iteration
8. `file_saved`: Boolean flag indicating whether code has been written to disk

Task 2: Implement Agent Nodes [40 marks]

Objective: Divide the system logic into specialized functions (nodes). Each node represents an autonomous agent or system process within the pipeline.

Subtask 2.1: Director and Architect Nodes

- **Function:** Handle initial user input and generate high-level system design.

Subtask 2.2: Engineer Node

- **Function:** Translate the architect's system design (and any subsequent QA feedback) into operational Python code.

Subtask 2.3: File I/O and Execution Nodes

- **Function:** Write the engineer's code to a local file and execute it via a subprocess, capturing any output or runtime errors. *This step requires a human-in-the-loop pause before execution.*

Subtask 2.4: QA and Scorer Nodes

- **Function:** Store the execution logs against the code and original requirements. Assign a qualitative metric (score) based on code health, bugs, and feature completion.

Task 3: Create Graph Structure [20 marks]

Objective: Map the flow of execution and define the conditional routing logic.

Requirement: Compile your nodes into a directed state graph.

- **Routing Logic:** Implement a conditional edge after the Scorer node. Based on a human intervention prompt, route the graph either to the END state, or loop it back to the Engineer node for code refinement.

Task 4: System Invocation [10 marks]

Objective: Execute the graph.

- **Requirement:** Stream the application execution in your terminal to observe and log real-time agent processing and state transitions.
- **Loop Session:** Implement a memory saver to keep historical context memorized in your application.

Task 5: Game Demo [20 marks]

Objective: Provide a working demonstration of the final, AI-generated Python game. You can execute the .py file saved in your workspace.

Game Requirements: The final product must successfully implement:

1. Flying obstacles (e.g., Pterodactyls).
2. Ground obstacles (e.g., Cacti).
3. Accurate jumping and falling physics.
4. Correct jumping and ducking mechanics mapped to user input.
5. A functional high-score tracking system.

Bonus Part [25 marks]:

Task 1: QA Node Subgraph Implementation

Objective: Instead of having a single `qa_node`, create and deploy a `QA_Subgraph` that internally routes data between a `syntax_checker_node`, a `logic_tester_node`, and a `performance_auditor_node` (scorer). This entire complex graph must simply appear as a single node to the parent graph.

Deliverables:

1. Template notebook renamed to `<roll_number>_pa4.ipynb`
2. Dino Runner code renamed to `<roll_number>_dino_runner.py`
3. A demo video showing workflow iterations and game execution.
4. Separate template notebook for EC part renamed to `<roll_number>_EC_pa4.ipynb`
5. All files zipped in a folder named `<roll_number>.zip`